

NiceGUI APIRouter 详细使用指南

NiceGUI 的 `APIRouter` 是用于**模块化管理 API 路由**的核心工具，可将不同业务模块的 API 接口拆分到不同文件 / 类中，避免主程序路由臃肿，提升代码的可维护性和复用性。本文从基础用法到高级场景，全面讲解其使用方式。

一、核心概念

- `APIRouter`：本质是一个「路由注册器」，可独立定义 GET/POST/PUT/DELETE 等 HTTP 方法的接口，最终挂载到主 NiceGUI 应用实例上。
- 路由挂载：将多个 `APIRouter` 实例（对应不同模块）注册到主应用，实现路由的模块化拆分。
- 路径前缀：可为每个 `APIRouter` 设置统一的路径前缀（如 `/api/v1/user`），简化子路由的定义。

二、基础使用步骤

1. 安装依赖

确保安装最新版 NiceGUI（`APIRouter` 从 NiceGUI 1.2+ 开始支持）：

```
pip install nicegui --upgrade
```

2. 最简示例：单文件路由拆分

步骤 1：创建 `APIRouter` 实例并定义接口

```
# main.py
from nicegui import APIRouter, app, ui

# 1. 创建 APIRouter 实例，可指定前缀（可选）
api_router = APIRouter(prefix="/api/v1")

# 2. 为 APIRouter 定义接口（支持 GET/POST/PUT/DELETE 等）
@api_router.get("/hello") # 最终路径: /api/v1/hello
async def hello_api(name: str = "Guest"):
    return {"message": f"Hello, {name}!"}

@api_router.post("/submit") # 最终路径: /api/v1/submit
async def submit_data(data: dict):
    # 处理 POST 请求体数据
    return {"status": "success", "received": data}

# 3. 将 APIRouter 挂载到主应用
app.include_router(api_router)

# 启动应用
if __name__ in {"__main__", "__mp_main__"}:
    ui.run()
```

步骤 2: 测试接口

- GET 请求: `http://localhost:8080/api/v1/hello?name=NiceGUI`

响应:

```
{"message": "Hello, NiceGUI!"}
```

- POST 请求: `http://localhost:8080/api/v1/submit`

请求体 (JSON) :

```
{"key": "value"}
```

响应:

```
{"status": "success", "received": {"key": "value"}}
```

三、进阶用法: 多模块拆分 (推荐)

实际项目中, 建议按业务模块 (如用户、订单、商品) 拆分 `APIRouter`, 结构如下:

```
project/
├─ main.py          # 主应用入口
├─ api/
│   ├─ __init__.py
│   ├─ user_router.py # 用户模块路由
│   └─ order_router.py # 订单模块路由
```

1. 定义模块级 APIRouter

(1) 用户模块路由 (api/user_router.py)

```
from nicegui import APIRouter

# 定义用户模块路由, 前缀 /api/v1/user
user_router = APIRouter(prefix="/api/v1/user")

# 用户列表接口: GET /api/v1/user/list
@user_router.get("/list")
async def get_user_list(page: int = 1, size: int = 10):
    # 模拟数据库查询
    return {
        "page": page,
        "size": size,
        "data": [{"id": 1, "name": "User1"}, {"id": 2, "name": "User2"}]
    }

# 创建用户接口: POST /api/v1/user/create
@user_router.post("/create")
async def create_user(user_info: dict):
```

```
# 模拟创建用户
return {"status": "success", "user_id": 1001}
```

(2) 订单模块路由 (api/order_router.py)

```
from nicegui import APIRouter

# 定义订单模块路由, 前缀 /api/v1/order
order_router = APIRouter(prefix="/api/v1/order")

# 获取订单详情: GET /api/v1/order/{order_id}
@order_router.get("/{order_id}")
async def get_order_detail(order_id: str):
    return {
        "order_id": order_id,
        "status": "paid",
        "amount": 99.9
    }

# 更新订单状态: PUT /api/v1/order/{order_id}/status
@order_router.put("/{order_id}/status")
async def update_order_status(order_id: str, status: str):
    return {"order_id": order_id, "new_status": status}
```

2. 主应用挂载所有路由 (main.py)

```
from nicegui import app, ui
from api.user_router import user_router
from api.order_router import order_router

# 挂载用户模块路由
app.include_router(user_router)
# 挂载订单模块路由
app.include_router(order_router)

# 可选: 添加前端页面 (非必须, APIRouter 仅处理 API)
@ui.page("/")
def home_page():
    ui.label("NiceGUI API Router Demo").classes("text-2xl")

if __name__ in {"__main__", "__mp_main__":
    ui.run()
```

3. 测试模块接口

接口路径	请求方法	示例参数	响应示例
/api/v1/user/list?page=1&size=5	GET	page=1, size=5	{"page":1,"size":5,"data":[...]}
/api/v1/user/create	POST	{"name":"NewUser"}	{"status":"success","user_id":1001}
/api/v1/order/123456	GET	order_id=123456	{"order_id":"123456","status":"paid"...

接口路径	请求方法	示例参数	响应示例
/api/v1/order/123456/status	PUT	status="shipped"	{"order_id": "123456", "new_status": "shipped"}

四、关键特性与高级配置

1. 路径参数与查询参数

`APIRouter` 支持 Flask/FastAPI 风格的参数解析：

- **路径参数**：通过 `{param}` 定义，自动解析为函数参数（支持 str/int/float 类型）。
- **查询参数**：函数参数自动从 URL 查询字符串中提取（支持默认值）。
- **请求体**：POST/PUT 请求的 JSON 数据自动解析为 dict 类型参数。

示例：

```
@api_router.get("/items/{item_id}")
async def get_item(item_id: int, category: str = "default"):
    # item_id 从路径提取, category 从查询参数提取
    return {"item_id": item_id, "category": category}
```

访问 `http://localhost:8080/api/v1/items/10?category=electronics`，响应：

```
{"item_id": 10, "category": "electronics"}
```

2. 中间件 (Middleware)

可为 `APIRouter` 添加专属中间件，实现权限校验、日志记录、请求拦截等：

```
from nicegui import APIRouter, Request, Response

api_router = APIRouter(prefix="/api/v1")

# 定义中间件函数
async def auth_middleware(request: Request, call_next):
    # 校验 Token
    token = request.headers.get("Authorization")
    if not token or token != "valid_token":
        return Response(status_code=401, content={"error": "Unauthorized"})
    # 执行下一个处理函数
    response = await call_next(request)
    return response

# 为路由添加中间件
api_router.add_middleware(auth_middleware)

# 受保护的接口
@api_router.get("/protected")
async def protected_api():
    return {"message": "This is a protected API"}
```

3. 响应状态码与自定义响应

可通过 `Response` 类自定义响应状态码、头部和内容：

```
from nicegui import APIRouter, Response

api_router = APIRouter(prefix="/api/v1")

@api_router.post("/create-item")
async def create_item(data: dict):
    if "name" not in data:
        # 返回 400 错误
        return Response(
            status_code=400,
            content={"error": "Name is required"},
            headers={"X-Error-Code": "MISSING_FIELD"}
        )
    # 返回 201 创建成功
    return Response(
        status_code=201,
        content={"status": "success", "item_id": 2001}
    )
```

4. 路由前缀叠加

若主应用挂载时指定前缀，会与 `APIRouter` 自身的前缀叠加：

```
# APIRouter 定义时的前缀
api_router = APIRouter(prefix="/user")

# 主应用挂载时额外添加前缀
app.include_router(api_router, prefix="/api/v1")

# 最终接口路径: /api/v1/user/list
@api_router.get("/list")
async def get_user_list():
    return {"data": []}
```

5. 跨域支持 (CORS)

若 API 供前端跨域调用，需配置 CORS 中间件：

```
from nicegui import app, APIRouter
from starlette.middleware.cors import CORSMiddleware

# 配置 CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # 允许所有来源（生产环境需指定具体域名）
    allow_credentials=True,
    allow_methods=["*"], # 允许所有 HTTP 方法
```

```
allow_headers=["*"], # 允许所有请求头
)

api_router = APIRouter(prefix="/api/v1")
# ... 定义接口 ...
app.include_router(api_router)
```

五、常见问题与注意事项

1. 路由冲突

若多个 `APIRouter` 定义了相同路径，后挂载的路由会覆盖先挂载的，需确保路径唯一（建议通过模块前缀隔离）。

2. 异步 / 同步函数

`APIRouter` 支持同步和异步函数，但建议使用 `async def`（异步）以提升并发性能。

3. 请求体解析

- POST/PUT 请求的 JSON 数据需通过 `Content-Type: application/json` 头部传递，否则无法解析为 dict。
- 表单数据（`application/x-www-form-urlencoded`）可通过 `request.form()` 获取：

```
from nicegui import APIRouter, Request

api_router = APIRouter(prefix="/api/v1")

@api_router.post("/form-submit")
async def form_submit(request: Request):
    form_data = await request.form()
    return {"username": form_data.get("username")}
```

4. 与 NiceGUI 页面路由的区别

- `APIRouter` 用于**后端 API 接口**（返回 JSON 等数据），基于 Starlette 实现。
- `@ui.page` 用于**前端页面路由**（返回 HTML 页面），是 NiceGUI 专属的页面注册方式。
- 两者可共存，但路径需避免冲突。

5. 部署注意事项

- 生产环境部署时，建议通过 `ui.run(host="0.0.0.0", port=80, production=True)` 启动。
- 若使用反向代理（如 Nginx），需确保路径转发正确，避免前缀丢失。

六、总结

NiceGUI 的 `APIRouter` 是模块化管理 API 的最佳实践，核心优势：

1. **拆分代码**：按业务模块拆分路由，避免主程序臃肿。
2. **灵活扩展**：支持中间件、自定义响应、参数解析等高级特性。
3. **兼容 Starlette**：底层基于 Starlette，可复用 Starlette/FastAPI 的生态。

适合场景：

- 构建前后端分离的 Web 应用。
- 开发 RESTful API 接口。
- 需模块化管理路由的中大型 NiceGUI 项目。

通过合理使用 `APIRouter`，可大幅提升 NiceGUI 项目的可维护性和扩展性。